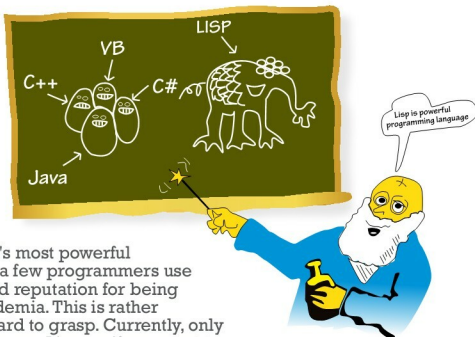


Lisp

Tears of Joy



Lisp has been hailed as the world's most powerful programming language. But only a few programmers use it because of its cryptic syntax and reputation for being appropriate for only those in academia. This is rather unfortunate, since Lisp isn't that hard to grasp. Currently, only the top percentile of programmers use Lisp; so if you want to be among the *crème de la crème*, these articles are for you.

Part—1

"I am going to hate my job." Those were my initial thoughts when I received an assignment at work, a few years ago. I had been asked to maintain and leverage a client module written in Lisp. My perception of Lisp was that of an ancient functional programming language with a cryptic syntax, used only by academicians and scientists to carry out experiments in the domain of Artificial Intelligence. And Lisp's ever-present parentheses were enough to drive anyone crazy!

At that time, I believed that I was an ace at a cool, new-age, object-oriented programming language, which was the medium of my expression: I ate, drank, slept, and dreamt in that language. It made me the God of my machine universe.

I also believed I was exceptionally attractive to women, and spent hours in front of the mirror doing my hair, and drove my sluggish and worn-out Kinetic Honda as if it was a 1340cc Suzuki Hayabusa.

I now know better...

Déjà vu

Those of us who witnessed the shift from the non-structured programming paradigm to procedural programming, and then towards object-oriented programming, will relate to what Paul Graham had to say in his book, 'Hackers & Painters'. It is something to the effect of, "You can't trust the opinion of others regarding which programming language will make you a better programmer. You're satisfied with whatever programming language you happen to use, because it dictates the way you think about programs. I know this from my own experience, as a high-school kid writing programs in BASIC. That language didn't even support recursion. It's hard to imagine writing programs without using recursion, but I didn't miss it at the time. I thought in BASIC. And I was a whiz at it. Master of all I surveyed."

Three weeks into hacking Lisp, I had a feeling of déjà vu—the previous experience being when I first 'progressed' from BASIC to C, and from C to C++ and Java. With each leap, I would be happily surprised with the growing power (of programming) at my fingertips. Time and again, I would wonder how I ever coded without objects, methods, encapsulation, polymorphism, inheritance, etcetera! One may say that it was syntactic sugar at work.

But not with Lisp, which is pure ecstasy. It's not just beautiful, but strangely beautiful.

In his famous essay, 'How to become a Hacker', Eric Steven Raymond (author of many best sellers including 'The Cathedral and the Bazaar'), writes: "Lisp is worth learning for the profound enlightenment experience you will have when you finally get it. That experience will make you a better programmer for the rest of your days, even if you never actually use Lisp itself, a lot."

Lisp enlightens you as a hacker

So what's so great about Lisp? How does it enlighten you as a hacker? Stephen Paul Graham explains this so proficiently and methodically that it will be inappropriate to answer this question in any other words than his. The five languages (Python, Java, C/C++, Perl, and Lisp) that Eric Raymond recommends to hackers fall at various points on the power continuum. Where they fall relative to one another is a sensitive topic, but I think Lisp is at the top. And to support this claim, I'll tell you about one of the things I find missing when I look at the other four languages. How can you get anything done in them without macros?

Many languages have something called a macro. But Lisp macros are unique. Lisp code is made out of Lisp data objects—and not in the trivial sense that the source files contain characters—and strings are one of the data types